# Studio Nx

P1-21 공지 API와 관리자 화면 — 함수 설명 합본 (백엔드 82~92)

2026-10-12

# 목차


1. ai-notes/backend(fastAPI_Python)/P1-21 - 공지 API와 관리자 화면/82_model_dev_note.md
2. ai-notes/backend(fastAPI_Python)/P1-21 - 공지 API와 관리자 화면/83_dev_note_list_published.md
3. ai-notes/backend(fastAPI_Python)/P1-21 - 공지 API와 관리자 화면/84_dev_note_get_published.md
4. ai-notes/backend(fastAPI_Python)/P1-21 - 공지 API와 관리자 화면/85_dev_notes_routes.md
5. ai-notes/backend(fastAPI_Python)/P1-21 - 공지 API와 관리자 화면/86_admin_auth_login.md
6. ai-notes/backend(fastAPI_Python)/P1-21 - 공지 API와 관리자 화면/87_admin_auth_logout_authenticate.md
7. ai-notes/backend(fastAPI_Python)/P1-21 - 공지 API와 관리자 화면/88_admin_dev_note_admin.md
8. ai-notes/backend(fastAPI_Python)/P1-21 - 공지 API와 관리자 화면/89_admin_dev_note_soft_delete.md
9. ai-notes/backend(fastAPI_Python)/P1-21 - 공지 API와 관리자 화면/90_admin_inquiry_admin.md
10. ai-notes/backend(fastAPI_Python)/P1-21 - 공지 API와 관리자 화면/91_admin_user_admin.md
11. ai-notes/backend(fastAPI_Python)/P1-21 - 공지 API와 관리자 화면/92_admin_setup_admin.md

# Backend(FastAPI/Python) 82 — `DevNote` 모델 (나머지 컬럼)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/models/dev_note.py` |
| 작업 | P1-21 공지 API와 관리자 화면: `dev_notes` 테이블과 짝이 되는 모델 채우기 (PRD 01 FR-114, FR-115) |
| 작성 | Claude (AI), 2026-10-13, 사용자 검토 |
| 이전 · 다음 | (P1-21 첫 항목) · 83 `list_published()` (다음) |

## 1. 하는 일

뼈대에는 `note_id`와 `author_id` 두 칸만 있었습니다. DB의 `dev_notes` 테이블(`DB.md` §3.1, `01_phase1_account.sql`)과 같도록 **나머지 8칸**을 채웠습니다. 이 모델을 서비스(83, 84)와 관리자 화면(86 이후)이 씁니다.

| 칸 | 타입 | 뜻 |
|---|---|---|
| `title` | `String(150)` | 제목 |
| `summary` | `String(300)` | 카드에 보이는 짧은 요약 |
| `content` | `MEDIUMTEXT` | 본문 (Markdown) |
| `is_published` | `Boolean`, 기본 0 | 공개 여부. 0은 초안이라 메인 페이지에 보이지 않습니다 |
| `published_at` | `DATETIME(3)`, 비어 있을 수 있음 | 공개 시각. 메인 페이지 정렬 기준이며, 미래로 두면 그때까지 숨는 예약 공개입니다 |
| `created_at` | `DATETIME(3)`, 기본 `CURRENT_TIMESTAMP(3)` | 만든 시각 |
| `updated_at` | `DATETIME(3)`, 기본 현재, 바뀔 때 갱신 | 고친 시각 |
| `deleted_at` | `DATETIME(3)`, 비어 있을 수 있음 | 소프트 삭제 (관리자 화면에서 지워도 행은 남습니다) |

## 2. 언제 쓰나

| 호출하는 곳 | 용도 |
|---|---|
| 83 `list_published()`, 84 `get_published()` | 공개된 공지를 읽을 때 |
| `DevNoteAdmin` (관리자 화면) | 공지를 작성, 수정, 공개할 때 |

## 3. 코드

### import

```
from sqlalchemy import BigInteger, Boolean, FetchedValue, ForeignKey, String, text
from sqlalchemy.dialects.mysql import DATETIME, MEDIUMTEXT
```

### 나머지 컬럼

```
title: Mapped[str] = mapped_column(String(150))
summary: Mapped[str] = mapped_column(String(300))
content: Mapped[str] = mapped_column(MEDIUMTEXT)
is_published: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
published_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
created_at: Mapped[datetime] = mapped_column(
    DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
)
updated_at: Mapped[datetime] = mapped_column(
    DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
)
deleted_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

### 문법 풀이

| 식 | 뜻 |
|---|---|
| `Mapped[str]` | 이 칸은 파이썬에서 글자(`str`)입니다. `NULL`이 안 되는 칸입니다 |
| `Mapped[datetime \| None]` | 날짜이거나 비어 있을 수 있는 칸입니다 (`NULL` 허용) |
| `String(150)` | 최대 150글자의 `VARCHAR(150)`입니다 |
| `MEDIUMTEXT` | 최대 약 16MB의 긴 글자 칸입니다. 일반 `Text`(`TEXT`)는 약 64KB라서 긴 공지 본문에는 모자랍니다 |
| `Boolean` | 파이썬에서는 `True`, `False`이고 MySQL에서는 `TINYINT(1)`입니다 |
| `server_default=text("0")` | 값을 안 넣으면 **DB가** 0을 넣습니다 |
| `DATETIME(fsp=3)` | 밀리초(소수점 3자리)까지 저장합니다 |
| `server_onupdate=FetchedValue()` | 행이 바뀌면 DB가 `updated_at`을 갱신하므로, 파이썬은 바뀐 값을 다시 읽어 오라는 표시입니다 |

## 4. 흐름

새 공지를 저장할 때 `title`, `summary`, `content`, `author_id`만 넣으면 `is_published`는 0(초안), `created_at`과 `updated_at`은 DB가 현재 시각으로 채웁니다. 공개하려면 `is_published`를 켜고 `published_at`을 정합니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| `is_published`를 `TINYINT(1)`이 아니라 `Boolean`으로 | 뼈대 주석의 지시이기도 하고, `TINYINT(1)`로 쓰면 값이 `0`, `1`로 나와서 `is False` 비교가 맞지 않습니다 (시험에서 확인했습니다). MySQL에는 같은 `TINYINT(1)`로 만들어집니다 |
| `content`를 `MEDIUMTEXT`로 | DB.md가 정한 대로이며, 공지는 Markdown 본문이 길어질 수 있습니다 |
| 시각 기본값을 DB가 채움 | 다른 모델(`Inquiry`, `ProjectSession`)과 같습니다. 연결 시간대를 UTC로 고정했으므로(P1-17) 저장되는 값은 UTC입니다 |
| `deleted_at` 소프트 삭제 | `DB.md`가 `dev_notes`를 소프트 삭제 대상으로 정했습니다 |
| 관계(`relationship`)는 만들지 않음 | 지금은 `author_id` 번호만 필요합니다. 작성자 이름을 보여 줄 일이 생기면 그때 더합니다 |

## 6. 시험 (`tests/test_dev_note_model.py`, 4개)

| 시험 | 결과 |
|---|---|
| 10칸이 실제 DB와 같음 (이름, 순서, `NULL` 허용, FK 1개) | 통과입니다 |
| 최소 칸만 넣고 저장하면 초안(`False`), 공개 시각과 삭제 시각이 비어 있음, `created_at`은 시간대 없는 값 | 통과입니다 |
| 약 20만 글자의 한글과 기호 본문, 150자 제목과 300자 요약, 밀리초 공개 시각이 그대로 저장됨 | 통과입니다 |
| 없는 작성자 번호는 DB가 거절(외래키) | 통과입니다 |

4개 모두 통과했습니다. 백엔드 전체는 **572개 통과**, `ruff`도 통과했습니다.

# Backend(FastAPI/Python) 83 — `list_published()` (공개된 공지 목록)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/services/dev_note_service.py` |
| 작업 | P1-21 공지 API와 관리자 화면: 메인 페이지에 보일 공개 공지 목록 만들기 (PRD 01 FR-114) |
| 작성 | Claude (AI), 2026-10-13, 사용자 검토 |
| 이전 · 다음 | 82 `DevNote` 모델 · 84 `get_published()` (다음) |

## 1. 하는 일

공개해도 되는 공지를 **최근 공개순**으로 `limit`개 돌려줍니다. 메인 페이지의 Development 카드는 `limit=4`로 부릅니다. 보이는 조건은 함수 `published_filters()`에 모았습니다 (84번 상세도 같은 조건을 씁니다).

| 공지의 상태 | 목록에 보임? | 이유 |
|---|---|---|
| 공개했고 공개 시각이 지남 | 보임 | 정상 |
| 초안 (`is_published = 0`) | 안 보임 | 아직 공개 전 |
| 지운 공지 (`deleted_at`이 있음) | 안 보임 | 소프트 삭제 |
| 공개 시각이 미래 | 안 보임 | 예약 공개, 그 시각이 되면 나타납니다 |
| 공개했는데 공개 시각이 비어 있음 | 안 보임 | 시각을 비교할 수 없어서 숨깁니다 (관리자 화면이 공개할 때 시각을 채웁니다) |

정렬은 `published_at` 내림차순이고, 시각이 같으면 `note_id`가 큰 것(나중에 만든 것)이 먼저입니다.

## 2. 언제 쓰나

| 호출하는 곳 | 화면 |
|---|---|
| 85 `GET /dev-notes` 라우터 (다음 이후) | 메인 페이지의 Development 카드 4개 (`home.js`의 `loadDevNotes`) |

## 3. 코드

### import

```
from datetime import UTC, datetime
from sqlalchemy import select
```

### 함수

```
def published_filters(now: datetime) -> list:
    return [
        DevNote.is_published.is_(True),
        DevNote.deleted_at.is_(None),
```

```
        DevNote.published_at <= now,
    ]


def list_published(db: Session, limit: int) -> list[DevNote]:
    now = datetime.now(UTC).replace(tzinfo=None)
    return list(
        db.scalars(
            select(DevNote)
            .where(*published_filters(now))
            .order_by(DevNote.published_at.desc(), DevNote.note_id.desc())
            .limit(limit)
        ).all()
    )
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

### 문법 풀이

| 식 | 뜻 |
|---|---|
| `datetime.now(UTC).replace(tzinfo=None)` | 지금의 UTC 시각을 시간대 표시 없이 만듭니다. DB에 저장된 값과 같은 기준입니다 |
| `DevNote.is_published.is_(True)` | SQL의 `is_published IS TRUE`입니다 |
| `DevNote.deleted_at.is_(None)` | SQL의 `deleted_at IS NULL`입니다 |
| `DevNote.published_at <= now` | SQL의 `published_at <= '현재 시각'`입니다. 칸이 `NULL`이면 결과가 참이 아니라서 제외됩니다 |
| `.where(*조건목록)` | 목록의 조건을 모두 `AND`로 묶어 적용합니다 |
| `.order_by(a.desc(), b.desc())` | `a` 내림차순, 같으면 `b` 내림차순입니다 |
| `.limit(limit)` | 최대 `limit`개만 가져옵니다 |
| `db.scalars(...).all()` | 한 줄에 모델 하나씩, 모두 가져옵니다 |

## 4. 흐름

1. 지금 시각(UTC)을 구합니다.
2. 공개되었고, 지워지지 않았고, 공개 시각이 지금 이전인 공지만 고릅니다.
3. 공개 시각이 최근인 순서로 정렬하고, `limit`개만 자릅니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 조건을 `published_filters()`로 따로 뺌 | 목록과 상세(84번)가 같은 기준이어야 "목록에는 없는데 주소를 치면 보이는" 일이 없습니다 |
| 예약 공개를 `published_at <= 지금`으로 처리 | 별도 스케줄러 없이도 시각이 되면 저절로 나타납니다 |
| 같은 시각은 `note_id` 내림차순 | 순서가 매번 같게 정해지고(안정적 정렬), 같은 시각에 올린 공지는 나중 것이 위에 옵니다 |
| 조건과 정렬이 `IDX(is_published, published_at)` 인덱스와 맞음 | 공지가 많아져도 빠르게 찾습니다 |

| 선택 | 이유 |
|---|---|
| 시각 비교를 DB(`NOW()`)가 아니라 파이썬의 UTC로 | 시험에서 시각을 바꿔 확인하기 쉽고, 저장 기준(UTC)과 같습니다 |
| `limit`의 범위 검사는 라우터가 함 | `limit`은 1~50이어야 하고, `Query(ge=1, le=50)`로 라우터에서 막습니다 (85번) |

## 6. 시험 (`tests/test_dev_note_service.py`, 4개)

| 시험 | 결과 |
|---|---|
| 초안, 지운 것, 미래 시각, 시각 없음은 빠지고 정상 공지만 나옴 | 통과입니다 |
| 최신순이고, 같은 시각이면 `note_id`가 큰 것이 먼저 | 통과입니다 |
| 6개를 만들고 `limit` 4, 1 | 4개, 1개 |
| 30초 뒤 공개로 예약한 공지는 안 보이다가, 시각이 지나면 보임 | 통과입니다 |

4개 모두 통과했습니다. `ruff`도 통과했습니다.

# Backend(FastAPI/Python) 84 — `get_published()` (공개된 공지 하나)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/services/dev_note_service.py` |
| 작업 | P1-21 공지 API와 관리자 화면: View More 모달이 보여 줄 공지 하나 가져오기 (PRD 01 FR-114) |
| 작성 | Claude (AI), 2026-10-13, 사용자 검토 |
| 이전 · 다음 | 83 `list_published()` · 85 `GET /dev-notes` 라우터 (다음) |

## 1. 하는 일

번호로 공지 하나를 찾아 돌려줍니다. 그 공지가 보이면 안 되는 상태이거나 없는 번호이면 **404**를 냅니다.

| 요청한 공지 | 결과 |
|---|---|
| 공개했고 공개 시각이 지남 | 공지를 돌려줌 |
| 초안 | 404 `NOT_FOUND` ("Note not found.") |
| 지운 공지 | 404 |
| 공개 시각이 미래 (예약 중) | 404 |
| 공개했는데 공개 시각이 비어 있음 | 404 |
| 없는 번호 | 404 |

## 2. 언제 쓰나

| 호출하는 곳 | 화면 |
|---|---|
| 85 `GET /dev-notes/{note_id}` 라우터 (다음) | 메인 페이지 Development 카드의 View More 모달 |

## 3. 코드

**import** (이번에 추가한 것만)

```
from app.core.errors import AppError, ErrorCode
```

**함수**

```
def get_published(db: Session, note_id: int) -> DevNote:
    now = datetime.now(UTC).replace(tzinfo=None)
    note = db.scalar(select(DevNote).where(DevNote.note_id == note_id, *published_filters(now)))
    if note is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Note not found.")
    return note
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

### 문법 풀이

| 식 | 뜻 |
|---|---|
| `db.scalar(select(...))` | 결과의 첫 행 하나를 가져옵니다. 없으면 `None`입니다 |
| `.where(번호 조건, *published_filters(now))` | 번호가 같다는 조건과, 83번에서 만든 "보여도 되는" 조건 세 개를 모두 `AND`로 묶습니다 |
| `raise AppError(코드, 404, 문구)` | 서비스가 일부러 내는 오류입니다. 오류 처리기가 `{"error": {"code": "NOT_FOUND", ...}}` JSON으로 바꿔 응답합니다 |

## 4. 흐름

1. 지금 시각(UTC)을 구합니다.
2. 번호가 같고 보여도 되는 조건(공개, 삭제 안 됨, 공개 시각이 지남)을 모두 만족하는 공지를 찾습니다.
3. 없으면 404를, 있으면 공지를 돌려줍니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 목록과 같은 조건(`published_filters`)을 씀 | 목록에 안 보이는 공지는 주소를 알아도 안 보여야 합니다. 초안 공지의 번호를 주소에 직접 치는 경우를 막습니다 |
| 없는 번호와 숨겨진 공지를 같은 404로 | "초안이 있다"는 사실도 알려 주지 않기 위해서입니다 |
| 먼저 찾고 나서 상태를 검사하지 않고 조건을 쿼리에 넣음 | 쿼리 한 번으로 끝나고, 초안의 내용이 파이썬 쪽으로 넘어올 일도 없습니다 |
| 문구는 "Note not found." 고정 | 번호나 내부 정보를 넣지 않습니다 |

## 6. 시험 (`tests/test_dev_note_service.py`, 이번에 7개 추가)

| 시험 | 결과 |
|---|---|
| 보이는 공지는 제목과 본문까지 돌려줌 | 통과입니다 |
| 초안, 지운 것, 예약 중, 공개 시각 없음, 없는 번호 5가지 | 모두 404, 코드 `NOT_FOUND`, 문구 `Note not found.` |
| 공지 여러 개를 두고, 목록에 있는 것만 상세로도 열리는지 비교 | 목록과 상세의 기준이 같음 |

이 파일의 시험은 83번의 4개와 합쳐 11개이며 모두 통과했습니다. `ruff`도 통과했습니다.

# Backend(FastAPI/Python) 85 — `GET /dev-notes`, `GET /dev-notes/{note_id}` 라우터

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/api/v1/dev_notes.py` |
| 작업 | P1-21 공지 API와 관리자 화면: 메인 페이지가 부르는 공지 API 주소 두 개 (PRD 01 FR-114) |
| 작성 | Claude (AI), 2026-10-13, 사용자 검토 |
| 이전 · 다음 | 84 `get_published()` · 86 `AdminAuth` (다음) |

## 1. 하는 일

83, 84번 서비스를 **API 주소**로 내보냅니다. 두 주소 모두 **로그인 없이** 읽을 수 있고, 쓰기는 없습니다 (작성은 관리자 화면에서만 합니다).

| 요청 | 응답 |
|---|---|
| `GET /api/v1/dev-notes` | 200 `{"items": [{note_id, title, summary, published_at}, …]}` 최대 4개 (기본) |
| `GET /api/v1/dev-notes?limit=10` | 최대 10개 (1~50, 밖이면 422 `VALIDATION_ERROR`) |
| `GET /api/v1/dev-notes/7` | 200 `{note_id, title, summary, published_at, content}` |
| 상세에서 초안, 지운 것, 예약 중, 없는 번호 | 404 `NOT_FOUND` |
| 상세에서 번호가 숫자가 아님 (`/dev-notes/abc`) | 422 `VALIDATION_ERROR` |

목록의 한 줄에는 **본문(`content`)이 없고**, 상세에만 있습니다. 카드 4개를 보여 줄 때 긴 본문을 함께 보내지 않기 위해서입니다.

## 2. 언제 쓰나

| 호출하는 곳 | 화면 |
|---|---|
| `pages/home.js`의 `loadDevNotes()` (아직 비어 있음) | 메인 페이지의 Development 카드, View More 모달 |

## 3. 코드

**import** (이번에 추가한 것만)

```
from app.schemas.dev_note import DEV_NOTES_LIMIT_MAX, DevNoteDetail, DevNoteItem, DevNoteListResponse
from app.services import dev_note_service
```

`router`, `Db`, `ErrorResponse`와 두 라우터의 모양(`response_model`, `limit` 범위)은 뼈대에 이미 있었습니다.

**함수**

```
@router.get("", response_model=DevNoteListResponse)
def list_dev_notes(db: Db, limit: Annotated[int, Query(ge=1, le=DEV_NOTES_LIMIT_MAX)] = 4):
    notes = dev_note_service.list_published(db, limit)
    return DevNoteListResponse(
        items=[DevNoteItem.model_validate(n, from_attributes=True) for n in notes]
    )


@router.get("/{note_id}", response_model=DevNoteDetail, responses={404: {"model": ErrorResponse}})
def read_dev_note(note_id: int, db: Db):
    return DevNoteDetail.model_validate(
        dev_note_service.get_published(db, note_id), from_attributes=True
    )
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

### 문법 풀이

| 식 | 뜻 |
|---|---|
| `Query(ge=1, le=50)` | 쿼리 `limit`이 1 이상 50 이하여야 합니다. 밖이면 FastAPI가 422를 냅니다 |
| `DevNoteItem.model_validate(n, from_attributes=True)` | DB 모델(`DevNote`)의 칸을 읽어 응답 모양(`DevNoteItem`)으로 바꿉니다. 응답 모양에 없는 칸(`content`, `author_id` 등)은 버려집니다 |
| `response_model=…` | 이 모양으로 응답을 한 번 더 걸러 냅니다 |
| `note_id: int` | 주소의 번호를 정수로 받으며, 숫자가 아니면 422입니다 |
| `db: Db` | 요청마다 DB 세션을 받습니다. 로그인한 회원(`CurrentUser`)은 받지 않으므로 토큰이 필요 없습니다 |

## 4. 흐름

1. 주소와 `limit`의 형식을 FastAPI가 먼저 검사합니다 (422).
2. 서비스(83 또는 84)가 공개된 공지를 찾습니다. 상세에서 못 찾으면 서비스가 404를 냅니다.
3. DB 모델을 응답 모양으로 바꿔 돌려줍니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 로그인 없이 | 메인 페이지는 로그인 전에도 보이므로, 공개 공지도 누구나 읽어야 합니다 (FR-114) |
| 목록과 상세의 응답 모양을 나눔 | 목록은 카드(날짜, 제목, 요약)만, 상세는 본문까지. 전송량이 줄어듭니다 |
| `content`는 Markdown 원문 그대로 | 화면(`home.js`)이 HTML로 바꿀 때 반드시 안전 처리(escape)해야 합니다. 서버는 원문을 건드리지 않습니다 |
| 라우터에는 판단을 두지 않음 | "무엇이 공개인가"는 서비스에만 있어서, 목록과 상세가 어긋나지 않습니다 |
| `author_id`는 응답에 없음 | 작성한 관리자의 번호를 공개할 이유가 없습니다 |

## 6. 시험 (`tests/test_dev_note_routes.py`, 13개)

| 시험 | 결과 |
|---|---|
| 두 주소가 앱에 등록됨 | 통과입니다 |
| 토큰 없이 목록 요청: 200, 한 줄에 네 칸(`note_id`, `title`, `summary`, `published_at`)뿐이고 `content` 없음 | 통과입니다 |
| 초안은 목록에 없고, 기본 개수는 4개 이하 | 통과입니다 |
| `limit`이 0, 51, -1, `abc` | 모두 422 `VALIDATION_ERROR` |
| 토큰 없이 상세 요청: 200, 본문(`content`) 포함, 다섯 칸 | 통과입니다 |
| 상세에서 초안, 지운 것, 예약 중, 없는 번호 | 모두 JSON 404 `NOT_FOUND` |
| 상세의 번호가 `abc` | 422 |

13개 모두 통과했습니다. 백엔드 전체는 **595개 통과**, `ruff`도 통과했습니다.

## 7. 남은 것

메인 페이지의 `loadDevNotes()`(JS)는 아직 비어 있습니다. P1-21은 백엔드 작업이고, 그 함수는 이 API가 준비된 뒤에 채우면 됩니다 (P1-19 이후 홈 화면 점검에서 정합니다).

# Backend(FastAPI/Python) 86 — `AdminAuth.login()` (관리자 화면 로그인)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/admin.py`, `requirements.txt` |
| 작업 | P1-21 공지 API와 관리자 화면: `/admin` 로그인 폼을 처리하는 함수 (PRD 01 FR-115) |
| 작성 | Claude (AI), 2026-10-13, 사용자 검토 |
| 이전 · 다음 | 85 공지 라우터 · 87 `AdminAuth.logout()`, `authenticate()` (다음) |

## 1. 하는 일

`/admin`에서 이메일과 비밀번호를 보내면, **ADMIN 계정인 경우에만** 로그인으로 인정하고 세션에 관리자 번호를 저장합니다.

| 입력 | 결과 |
|---|---|
| ADMIN 계정, 비밀번호 맞음 | 성공 (`True`), 세션에 `{"admin_user_id": 번호}`만 저장 |
| 일반 회원(`USER`), 비밀번호 맞음 | 실패 (`False`), 세션은 비어 있음 |
| 비밀번호 틀림, 없는 이메일, 빈 폼 | 실패 |
| 정지(`SUSPENDED`) 또는 탈퇴(`DELETED`)한 관리자 | 실패 |
| 이메일에 공백이나 대문자가 섞임 | 일반 로그인처럼 정리해서 확인 (성공 가능) |

실패 이유는 화면에 알려 주지 않습니다. 모두 같은 "잘못됨" 한 가지로 보입니다.

## 2. 언제 쓰나

| 호출하는 곳 | 화면 |
|---|---|
| SQLAdmin이 `/admin/login` 폼 전송을 받을 때 | 관리자 로그인 화면 |

## 3. 코드

**import** (이번에 추가한 것)

```
from app.core.errors import AppError
from app.db.session import SessionLocal
from app.models.user import UserRole
from app.services import auth_service
```

**함수**

```
async def login(self, request: Request) -> bool:
    form = await request.form()
    email = str(form.get("username") or "")
    password = str(form.get("password") or "")
```

```
    db = SessionLocal()
    try:
        user = auth_service.authenticate(db, email, password)
        if user.role != UserRole.ADMIN:
            return False
        request.session.update({"admin_user_id": user.user_id})
        return True
    except AppError:
        return False
    finally:
        db.close()
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

**패키지**: `requirements.txt`에 `itsdangerous>=2.1`을 추가했습니다. 세션 쿠키를 서명하는 데 쓰는 패키지이며, 없으면 `AdminAuth`를 만드는 순간 `ModuleNotFoundError`가 납니다. **다른 컴퓨터에서는 pull 뒤에 `pip install -r requirements-dev.txt`를 다시 실행해야 합니다.**

### 문법 풀이

| 식 | 뜻 |
|---|---|
| `await request.form()` | 로그인 폼에서 보낸 값을 읽습니다. SQLAdmin의 로그인 폼은 입력 칸 이름이 `username`, `password`입니다 |
| `str(form.get("username") or "")` | 칸이 없거나 비어 있어도 빈 글자로 바꿔서, 파일이나 `None`이 들어와도 오류가 나지 않게 합니다 |
| `SessionLocal()` | 이 함수가 직접 DB 세션을 엽니다. SQLAdmin은 FastAPI의 `Depends`를 쓰지 않아서 `Db`를 받을 수 없습니다 |
| `auth_service.authenticate(db, email, password)` | 일반 로그인(P1-05)과 같은 확인입니다. 이메일 정리, 비밀번호 확인, 탈퇴와 정지 계정 거절, 마지막 로그인 시각 기록까지 합니다 |
| `except AppError` | 확인에 실패하면 서비스가 `AppError`를 던지므로, 모두 `False`로 바꿉니다 |
| `finally: db.close()` | 성공하든 실패하든 DB 세션을 닫습니다. `return`이 `try` 안에 있어도 `finally`는 실행됩니다 |
| `request.session.update({...})` | SQLAdmin이 서명해서 쿠키로 보관하는 세션에 값을 저장합니다 |

## 4. 흐름

1. 폼에서 이메일과 비밀번호를 읽습니다.
2. 일반 로그인과 같은 `authenticate`로 확인합니다. 실패하면 `AppError`이므로 `False`입니다.
3. 확인된 회원의 `role`이 `ADMIN`이 아니면 `False`입니다.
4. 맞으면 세션에 `admin_user_id`를 저장하고 `True`입니다.
5. 어느 경우든 DB 세션을 닫습니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 새로 확인 코드를 만들지 않고 `authenticate`를 재사용 | 비밀번호 확인, 없는 이메일의 응답 시간 맞추기, 탈퇴와 정지 거절을 한 곳에서 관리합니다. 관리자 로그인에서만 다르게 동작할 이유가 없습니다 |

| 선택 | 이유 |
|---|---|
| 일반 회원은 비밀번호가 맞아도 `False` | 비밀번호가 맞다는 것과 관리자라는 것은 다릅니다. 권한은 `role`로 따로 확인합니다 |
| 실패 이유를 구분해서 알리지 않음 | "이 이메일은 관리자가 아님" 같은 정보로 관리자 계정을 찾지 못하게 합니다 |
| 세션에 번호 하나만 저장 | 비밀번호나 역할을 쿠키에 넣지 않습니다. 매 요청마다 DB에서 다시 확인합니다 (87번 `authenticate`) |
| API의 JWT와 세션 쿠키를 따로 씀 | 관리자 화면은 SQLAdmin이 자체 세션으로 관리하고, API의 토큰과 섞이지 않습니다 |
| 관리자 계정은 DB에서 직접 `role = 'ADMIN'`으로 지정 | 화면에서 관리자를 만드는 기능이 없습니다 (뼈대 설명 그대로) |

## 6. 시험 (`tests/test_admin_auth.py`, 7개)

| 시험 | 결과 |
|---|---|
| ADMIN 계정: `True`, 세션이 `{"admin_user_id": 번호}` 하나뿐 | 통과입니다 |
| 이메일에 공백과 대문자가 섞여도 성공 | 통과입니다 |
| 일반 회원은 비밀번호가 맞아도 `False`, 세션은 비어 있음 | 통과입니다 |
| 틀린 비밀번호, 없는 이메일, 빈 폼 | 모두 `False` |
| 정지된 관리자, 탈퇴한 관리자 | 모두 `False`, 세션은 비어 있음 |
| `login`이 DB 세션을 직접 열고 한 번 닫음 | 통과입니다 |

7개 모두 통과했습니다. `ruff`도 통과했습니다.

# Backend(FastAPI/Python) 87 — `AdminAuth.logout()`, `authenticate()` (로그아웃, 매 요청 확인)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/admin.py` |
| 작업 | P1-21 공지 API와 관리자 화면: `/admin` 로그아웃과 요청마다 하는 로그인 확인 (PRD 01 FR-115) |
| 작성 | Claude (AI), 2026-10-13, 사용자 검토 |
| 이전 · 다음 | 86 `AdminAuth.login()` · 88 `DevNoteAdmin` (다음) |

## 1. 하는 일

`AdminAuth`의 나머지 두 함수입니다. 서로 짝이라 함께 구현했습니다.

| 함수 | 언제 불리나 | 하는 일 |
|---|---|---|
| `logout` | 관리자 화면의 Logout 버튼 | 세션을 비우고 `True` |
| `authenticate` | `/admin`의 **모든 요청마다** | 지금 요청이 로그인한 관리자의 것인지 `True`, `False`로 답합니다. `False`이면 SQLAdmin이 로그인 화면으로 보냅니다 |

`authenticate`가 `True`가 되는 조건은 이렇습니다.

| 세션과 DB의 상태 | 결과 |
|---|---|
| 세션에 `admin_user_id`가 없음 (로그인 안 함) | `False` |
| 있고, DB에서 그 회원이 `ACTIVE`이며 `ADMIN` | `True` (세션 유지) |
| 로그인한 뒤에 `role`을 `USER`로 바꿈 (권한 회수) | `False`, 남은 세션도 비움 |
| 로그인한 뒤에 정지(`SUSPENDED`)나 탈퇴(`DELETED`) | `False`, 세션도 비움 |
| 세션의 번호가 DB에 없음 | `False` |
| 세션의 번호가 일반 회원 | `False` |

## 2. 언제 쓰나

| 호출하는 곳 | 화면 |
|---|---|
| SQLAdmin의 모든 `/admin/…` 요청 (`authenticate`), Logout 버튼 (`logout`) | 관리자 화면 전체 |

## 3. 코드

`import`는 86번에서 모두 추가했습니다.

**함수**

```
async def logout(self, request: Request) -> bool:
    request.session.clear()
```

```
    return True


async def authenticate(self, request: Request) -> bool:
    user_id = request.session.get("admin_user_id")
    if user_id is None:
        return False
    db = SessionLocal()
    try:
        user = auth_service.get_active_user(db, user_id)
        if user.role == UserRole.ADMIN:
            return True
    except AppError:
        pass
    finally:
        db.close()
    request.session.clear()
    return False
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

### 문법 풀이

| 식 | 뜻 |
|---|---|
| `request.session.clear()` | 세션의 모든 값을 지웁니다. 이미 비어 있어도 오류가 나지 않습니다 |
| `request.session.get("admin_user_id")` | 세션에서 값을 꺼냅니다. 없으면 `None`입니다 |
| `auth_service.get_active_user(db, user_id)` | 그 번호의 회원이 아직 `ACTIVE`이면 돌려주고, 없거나 정지, 탈퇴이면 `AppError`를 던집니다. 일반 API의 재발급 확인과 같은 함수입니다 |
| `try … except AppError: pass … finally` | 못 쓰는 계정이면 오류를 삼키고, 뒤쪽의 `clear()`와 `return False`로 갑니다. `return True`가 `try` 안에 있어도 `finally`의 `db.close()`는 실행됩니다 |

## 4. 흐름

`authenticate`:

1. 세션에 관리자 번호가 없으면 바로 `False`입니다.
2. 있으면 DB 세션을 열어 그 회원이 정상 계정인지 확인합니다.
3. 정상이면서 `ADMIN`이면 `True`입니다.
4. 그 밖의 모든 경우(정지, 탈퇴, 권한 회수, 없는 번호)는 세션을 비우고 `False`입니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 세션만 믿지 않고 **매 요청 DB 확인** | 세션 쿠키는 로그인 때의 상태를 담고 있습니다. 로그인한 뒤에 정지되거나 권한이 회수된 계정이 쿠키만으로 계속 관리자 화면을 쓰는 일을 막습니다. 요청 한 번에 쿼리 한 번(기본키 조회)이라 부담이 작습니다 |
| `get_active_user` 재사용 | 정지, 탈퇴 판단이 일반 API와 같은 기준이어야 합니다 |

| 선택 | 이유 |
|---|---|
| 못 쓰는 계정이면 세션도 비움 | 이후 요청에서 같은 확인을 반복하지 않고, 남은 쿠키가 쓸모없게 됩니다 |
| `logout`이 항상 `True` | 로그인하지 않은 채 불러도 문제가 없고, 실패할 일이 없습니다 |
| 세션에 역할을 저장하지 않음 | 역할은 DB가 정한 값이어서, 매번 DB에서 읽는 것이 맞습니다 |

## 6. 시험 (`tests/test_admin_auth.py`, 이번에 8개 추가)

| 시험 | 결과 |
|---|---|
| `logout`: 세션이 비워지고, 이미 비어 있어도 `True` | 통과입니다 |
| 세션이 없으면 `False` | 통과입니다 |
| 정상 관리자는 `True`이고 세션 유지 | 통과입니다 |
| 로그인한 뒤에 `role`을 `USER`로 바꾸면 다음 요청이 `False`, 세션도 비워짐 | 통과입니다 |
| 로그인한 뒤에 정지, 탈퇴 | 둘 다 `False`, 세션도 비워짐 |
| 없는 번호, 일반 회원 번호가 세션에 있음 | 둘 다 `False` |

이 파일의 시험은 86번의 7개와 합쳐 15개이며 모두 통과했습니다. `ruff`도 통과했습니다.

# Backend(FastAPI/Python) 88 — `DevNoteAdmin` (공지 관리 화면 설정)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/admin.py` |
| 작업 | P1-21 공지 API와 관리자 화면: `/admin`에서 공지를 쓰고 공개하는 화면의 설정 (PRD 01 FR-114, FR-115) |
| 작성 | Claude (AI), 2026-10-13, 사용자 검토 |
| 이전 · 다음 | 87 `AdminAuth` 로그아웃, 확인 · 89 소프트 삭제 (다음) |

## 1. 하는 일

SQLAdmin이 `DevNote` 모델로 만드는 화면을 공지 관리에 맞게 설정합니다.

| 영역 | 설정 |
|---|---|
| 목록 열 | 번호, 제목, 공개 여부, 공개 시각, 만든 시각 |
| 목록 정렬, 검색 | 번호 큰 것(최근)이 위, 제목으로 검색 |
| 작성, 수정 폼 | 제목, 요약, 본문, 공개 여부, 공개 시각 **다섯 칸만** (작성자, 만든 시각, 고친 시각, 삭제 시각은 폼에 없음) |
| 새 공지를 저장할 때 | 작성자(`author_id`)를 **로그인한 관리자**로 자동 입력 |
| 공개로 바꿨는데 공개 시각이 비어 있을 때 | **지금 시각**(UTC)을 자동 입력 |
| 본문(`content`) 입력칸 | **여러 줄 입력칸**(textarea). 실제 화면 시험에서 한 줄 입력창으로 나오는 것을 발견해 고쳤습니다 |
| 지운 공지(`deleted_at`이 있음) | 목록, 상세, 수정 화면에서 보이지 않음 |

## 2. 언제 쓰나

| 호출하는 곳 | 화면 |
|---|---|
| SQLAdmin이 `setup_admin()`으로 등록된 뒤 (92번) `/admin/dev-note/…` 화면 | 공지 목록, 새 공지, 수정 |

## 3. 코드

**import** (이번에 추가한 것)

```
from datetime import UTC, datetime
from wtforms import TextAreaField
```

**설정과 함수**

```
class DevNoteAdmin(ModelView, model=DevNote):
    name = "Dev note"
    name_plural = "Dev notes"
```

```
    column_list = [DevNote.note_id, DevNote.title, DevNote.is_published,
                   DevNote.published_at, DevNote.created_at]
    column_default_sort = [("note_id", True)]
    column_searchable_list = [DevNote.title]
    form_columns = [DevNote.title, DevNote.summary, DevNote.content,
                    DevNote.is_published, DevNote.published_at]
    form_overrides = {"content": TextAreaField}

    def list_query(self, request):
        return super().list_query(request).where(DevNote.deleted_at.is_(None))

    def count_query(self, request):
        return super().count_query(request).where(DevNote.deleted_at.is_(None))

    def form_details_query(self, request):
        return super().form_details_query(request).where(DevNote.deleted_at.is_(None))

    def form_edit_query(self, request):
        return super().form_edit_query(request).where(DevNote.deleted_at.is_(None))

    async def on_model_change(self, data, model, is_created, request):
        if is_created:
            data["author_id"] = request.session.get("admin_user_id")
        if data.get("is_published") and not data.get("published_at"):
            data["published_at"] = datetime.now(UTC).replace(tzinfo=None)
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

## 문법 풀이

| 식 | 뜻 |
|---|---|
| `class DevNoteAdmin(ModelView, model=DevNote)` | `DevNote` 모델의 관리 화면을 만드는 SQLAdmin 클래스입니다 |
| `form_overrides = {"content": TextAreaField}` | `content` 칸의 입력 방식을 여러 줄 입력칸으로 바꿉니다. SQLAdmin은 `MEDIUMTEXT`를 한 줄 입력창(`<input type="text">`)으로 만들기 때문입니다 |
| `column_list`, `form_columns` | 목록에 보일 열, 폼에 넣을 칸입니다. 여기에 없는 칸은 폼에서 나오지 않습니다 |
| `column_default_sort = [("note_id", True)]` | `note_id` 내림차순(`True` = 내림차순)입니다 |
| `list_query`, `count_query` | 목록과 전체 개수를 세는 쿼리입니다. 같은 조건을 둘 다 걸어야 쪽 번호가 맞습니다 |
| `form_details_query`, `form_edit_query` | 상세와 수정 화면이 공지 하나를 찾는 쿼리입니다. 지운 공지를 주소로 직접 열지 못하게 같은 조건을 겁니다 |
| `super().list_query(request).where(...)` | 기본 쿼리에 조건을 더합니다 |
| `on_model_change(data, model, is_created, request)` | 저장 직전에 불립니다. **`data`(폼 값)를 고치면** 그 값이 모델에 들어갑니다. 새 공지일 때 `model`은 아직 빈 상태입니다 |
| `request.session.get("admin_user_id")` | 86번 로그인이 저장한 관리자 번호입니다 |

## 4. 흐름

공지를 새로 쓰고 저장할 때:

1. 폼의 다섯 칸 값이 `data`로 들어옵니다.
2. `on_model_change`가 `data`에 작성자(로그인한 관리자)를 넣습니다.
3. 공개를 켜고 공개 시각이 비어 있으면 지금 시각을 넣습니다.
4. SQLAdmin이 `data`를 모델에 넣고 저장합니다.

수정할 때는 작성자를 바꾸지 않고, 공개 시각만 위 규칙대로 채웁니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 작성자를 폼에서 받지 않고 로그인한 관리자로 | 다른 사람 이름으로 쓰는 일을 막습니다 |
| 수정할 때는 작성자를 그대로 | 처음 쓴 사람이 기록으로 남아야 합니다 |
| 공개 시각이 비면 지금 시각 | 83번 목록이 `published_at <= 지금`인 공지만 보여 주므로, 시각이 비면 공개해도 안 보입니다. 관리자가 시각을 따로 안 정해도 바로 보이게 합니다 |
| 공개 시각을 직접 입력하면 그대로 (미래 포함) | 예약 공개를 쓸 수 있습니다 |
| 초안(공개 안 함)에는 시각을 넣지 않음 | 나중에 공개할 때 그 시점의 시각이 들어가야 합니다 |
| 모델이 아니라 `data`를 고침 | SQLAdmin이 `on_model_change` 뒤에 `data`를 모델에 덮어쓰기 때문에, 모델에 먼저 넣으면 사라집니다 (SQLAdmin 0.32의 `_queries.py`에서 순서를 확인했습니다) |
| 지운 공지를 네 쿼리에서 모두 제외 | 목록에서만 빼면 주소로 직접 열고 고칠 수 있습니다 |
| 본문만 `TextAreaField`로 바꿈 | 제목과 요약은 한 줄이면 되지만, Markdown 본문은 줄바꿈이 필요합니다 |
| 요약(`summary`)과 본문(`content`)을 폼에 둠 | 메인 페이지 카드와 View More 모달이 각각 씁니다 |

## 6. 시험 (`tests/test_admin_dev_note.py`, 9개)

| 시험 | 결과 |
|---|---|
| 폼의 칸이 정확히 다섯 개이고, 작성자, 시각 세 개, 번호는 없음 | 통과입니다 |
| 본문 칸이 `TextAreaField`로 바뀌어 있음 | 통과입니다 |
| 목록 열 다섯 개, 번호 내림차순, 제목 검색 | 통과입니다 |
| 목록, 개수, 상세, 수정 쿼리 네 개에 모두 `deleted_at IS NULL` | 통과입니다 |
| 새 공지: 로그인한 관리자 번호가 `author_id`로 들어감 | 통과입니다 |
| 수정: `author_id`를 건드리지 않음 | 통과입니다 |
| 공개로 저장하고 시각이 비어 있으면: 지금 UTC 시각(시간대 없음)이 들어감 | 통과입니다 |
| 직접 입력한 시각(미래 포함)은 그대로 | 통과입니다 |

| 시험 | 결과 |
| --- | --- |
| 초안은 시각이 비어 있는 그대로 | 통과입니다 |

9개 모두 통과했습니다. `ruff`도 통과했습니다. SQLAdmin 화면에서 실제로 쓰고 공개하는 흐름은 `setup_admin()`(92번)까지 구현한 뒤 서버를 띄워 확인합니다.

# Backend(FastAPI/Python) 89 — `DevNoteAdmin.delete_model()` (소프트 삭제)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/admin.py` |
| 작업 | P1-21 공지 API와 관리자 화면: 삭제 버튼이 행을 지우지 않고 `deleted_at`만 채우게 하기 (PRD 01 FR-115, `DB.md` 소프트 삭제 규칙) |
| 작성 | Claude (AI), 2026-10-13, 사용자 검토 |
| 이전 · 다음 | 88 `DevNoteAdmin` · 90 `InquiryAdmin` (다음) |

## 1. 하는 일

관리자 화면에서 공지의 삭제 버튼을 누르면, 행을 지우는 대신 **`deleted_at`에 지금 시각(UTC)을 넣습니다.** `DB.md`가 `dev_notes`를 소프트 삭제 대상으로 정했기 때문입니다.

| 상황 | 결과 |
|---|---|
| 공지를 삭제함 | `deleted_at`이 채워지고 행은 DB에 남습니다 |
| 삭제한 공지 | 관리자 목록, 상세, 수정 화면(88번)과 메인 페이지 공지 API(83, 84번)에서 모두 사라집니다 |
| 이미 지운 공지를 다시 삭제 | 아무것도 바꾸지 않습니다 (처음 지운 시각이 남음) |
| 없는 번호를 삭제 | 오류 없이 끝납니다 |
| 되살리기 | DB에서 `deleted_at`을 비우면 목록에 다시 나타납니다 |

## 2. 언제 쓰나

| 호출하는 곳 | 화면 |
|---|---|
| SQLAdmin의 삭제 요청 (`/admin/dev-note/delete`) | 공지 목록의 삭제 버튼, 여러 개를 골라 지우기 |

## 3. 코드

**import** (이번에 추가한 것)

```
from starlette.concurrency import run_in_threadpool
```

**함수**

```
async def delete_model(self, request: Request, pk) -> None:
    await run_in_threadpool(self._soft_delete, pk)

def _soft_delete(self, pk) -> None:
    with self.session_maker() as session:
        note = session.get(DevNote, int(pk))
        if note is not None and note.deleted_at is None:
```

```
            note.deleted_at = datetime.now(UTC).replace(tzinfo=None)
            session.commit()
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

### 문법 풀이

| 식 | 뜻 |
|---|---|
| `delete_model(self, request, pk)` | SQLAdmin이 삭제할 때 부르는 함수입니다. 기본 동작(`session.delete`)을 바꿔 덮어씁니다 |
| `run_in_threadpool(함수, 값)` | 일반(동기) 함수를 서버가 멈추지 않도록 별도 스레드에서 실행하고 기다립니다. 이 프로젝트의 DB 연결은 동기 방식이기 때문입니다 |
| `with self.session_maker() as session` | SQLAdmin이 `Admin(...)`을 만들 때 넣어 주는 DB 세션 공장입니다. 블록이 끝나면 세션을 닫습니다 |
| `session.get(DevNote, int(pk))` | 기본키로 공지 하나를 가져옵니다. `pk`는 글자로 오므로 정수로 바꿉니다 |
| `if note is not None and note.deleted_at is None` | 있고 아직 안 지운 공지일 때만 바꿉니다 |

## 4. 흐름

1. SQLAdmin이 삭제 요청을 받고, 먼저 그 공지가 있는지 확인한 뒤 `delete_model`을 부릅니다.
2. 공지를 찾아서 아직 안 지웠으면 `deleted_at`에 지금 시각을 넣고 저장합니다.
3. SQLAdmin이 목록으로 돌아가며, 이미 `deleted_at`이 있어서 그 공지는 보이지 않습니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 행을 지우지 않음 | 실수로 지워도 `deleted_at`을 비우면 되살릴 수 있고, `DB.md`의 소프트 삭제 규칙을 따릅니다 |
| 이미 지운 공지는 건드리지 않음 | 처음 지운 시각이 기록으로 남습니다. 같은 요청이 두 번 와도 결과가 같습니다 |
| 없는 번호는 오류 없이 | 삭제는 "그 상태가 되면 됨"이어서, 이미 없으면 목적이 달성된 것입니다 |
| `can_delete = False`로 버튼을 없애지 않음 | 관리자에게는 "삭제" 버튼이 자연스럽고, 안에서만 소프트 삭제로 바꿉니다 |
| 동기 세션을 스레드에서 실행 | `async` 함수 안에서 동기 DB를 직접 부르면 그동안 서버가 멈춥니다. SQLAdmin 자신도 같은 방식으로 합니다 |
| 이 함수에서 `on_model_delete` 같은 후크를 부르지 않음 | 지금 쓰는 후크가 없습니다 |

## 6. 시험 (`tests/test_admin_dev_note.py`, 이번에 5개 추가)

| 시험 | 결과 |
|---|---|
| 삭제하면 `deleted_at`이 채워지고(UTC, 시간대 없음) 행은 그대로 있음 | 통과입니다 |
| 공개된 공지를 지우면 메인 페이지 서비스(83 목록, 84 상세)에서 사라지고, 상세는 404 | 통과입니다 |
| 두 번 지워도 처음 지운 시각이 그대로 | 통과입니다 |

| 시험 | 결과 |
| --- | --- |
| 없는 번호(`999999999`)를 지워도 오류 없음 | 통과입니다 |
| `deleted_at`을 비우면 다시 목록에 나타남 (되살리기) | 통과입니다 |

이 파일의 시험은 88번의 9개와 합쳐 14개이며 모두 통과했습니다. `ruff`도 통과했습니다. SQLAdmin 화면의 삭제 버튼을 실제로 누르는 확인은 `setup_admin()`(92번) 이후 서버를 띄워서 합니다.

# Backend(FastAPI/Python) 90 — `InquiryAdmin` (문의 확인, 답변 화면 설정)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/admin.py` |
| 작업 | P1-21 공지 API와 관리자 화면: `/admin`에서 문의를 읽고 답변을 기록하는 화면의 설정 (PRD 01 FR-115) |
| 작성 | Claude (AI), 2026-10-13, 사용자 검토 |
| 이전 · 다음 | 89 소프트 삭제 · 91 `UserAdmin` (다음) |

## 1. 하는 일

메인 페이지의 Contact 폼으로 들어온 문의를 관리자가 확인하고 답변을 적는 화면입니다.

| 영역 | 설정 |
|---|---|
| 만들기, 삭제 | **둘 다 없음.** 문의는 Contact 폼으로만 들어오고, 오래된 문의는 보관 기간(1년)이 지나면 정리 작업(`purge_old_inquiries`)이 지웁니다 |
| 목록 열 | 번호, 이메일, 제목, 상태, 받은 시각. 최근 것이 위 |
| 검색 | 이메일, 제목 |
| 상세 | 이름, 이메일, 제목, 본문, 개인정보 동의 시각 등 모든 칸을 볼 수 있음 |
| 수정 폼 | **답변(`answer`)과 상태(`status`) 두 칸만** |
| 답변 저장 | 답변이 새로 쓰였거나 바뀌면 `answered_at`을 지금 시각(UTC)으로, 상태가 아직 `RECEIVED`이면 `ANSWERED`로 |
| 답변을 비움 | `answered_at`도 비움 (상태는 그대로) |

## 2. 언제 쓰나

| 호출하는 곳 | 화면 |
|---|---|
| `setup_admin()`으로 등록된 뒤 (92번) `/admin/inquiry/…` | 문의 목록, 상세, 답변 수정 |

## 3. 코드

**import** (이번에 추가한 것)

```
from app.models.inquiry import InquiryStatus
```

### 설정과 함수

```
class InquiryAdmin(ModelView, model=Inquiry):
    can_create = False
    can_delete = False
    column_list = [Inquiry.inquiry_id, Inquiry.email, Inquiry.title,
                   Inquiry.status, Inquiry.created_at]
    column_default_sort = [("inquiry_id", True)]
```

```
    column_searchable_list = [Inquiry.email, Inquiry.title]
    form_columns = [Inquiry.answer, Inquiry.status]

    async def on_model_change(self, data, model, is_created, request):
        answer = (data.get("answer") or "").strip()
        if not answer:
            data["answered_at"] = None
        elif answer != (model.answer or "").strip():
            data["answered_at"] = datetime.now(UTC).replace(tzinfo=None)
            if str(getattr(data.get("status"), "value", data.get("status"))) == "RECEIVED":
                data["status"] = InquiryStatus.ANSWERED
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

### 문법 풀이

| 식 | 뜻 |
|---|---|
| `can_create = False`, `can_delete = False` | 화면의 `New`, 삭제 버튼과 그 주소를 막습니다 |
| `form_columns = [answer, status]` | 수정 폼에 이 두 칸만 나옵니다. 이름, 이메일, 본문은 바꿀 수 없습니다 |
| `(data.get("answer") or "").strip()` | 답변이 `None`이거나 비어 있어도 빈 글자로 바꾸고 앞뒤 공백을 지웁니다 |
| `model.answer` | `on_model_change`가 불릴 때 `model`은 **저장돼 있던 문의 그대로**입니다. 그래서 이전 답변과 비교할 수 있습니다 |
| `data["answered_at"] = …` | 폼에 없는 칸이어도 `data`에 넣으면 SQLAdmin이 모델에 함께 넣어 저장합니다 |
| `getattr(data.get("status"), "value", data.get("status"))` | 상태가 열거형 값이든 평범한 글자든 같은 글자(`"RECEIVED"`)로 읽습니다 |

## 4. 흐름

답변을 쓰고 저장할 때:

1. 폼의 값(답변, 상태)이 `data`로 들어옵니다.
2. 답변이 비어 있으면 `answered_at`을 비웁니다.
3. 답변이 이전과 다르면 `answered_at`을 지금으로 넣고, 상태가 `RECEIVED`이면 `ANSWERED`로 바꿉니다.
4. 답변이 이전과 같으면(상태만 바꾸는 경우 등) `answered_at`을 그대로 둡니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| `answered_at`을 직접 입력하지 않고 자동으로 | 답변 시각은 정리 작업의 **보관 기간 기준**(답변 후 1년)이어서, 정확하게 기록되어야 합니다 |
| 답변이 **바뀐 경우에만** 시각 갱신 | 상태만 바꾸거나 같은 답변을 다시 저장할 때 보관 기간이 새로 시작되지 않게 합니다 |
| 답변을 쓰면 상태를 자동으로 `ANSWERED` | 답변을 쓰고 상태를 바꾸는 걸 잊는 일을 줄입니다. 이미 `CLOSED` 등으로 정했으면 그 선택을 존중합니다 |
| 답변을 비우면 시각도 비움 | 답변이 없는데 답변 시각만 남는 모순을 막습니다 |
| 삭제 버튼 없음 | 답변 기록을 실수로 지우지 않게 하고, 지우는 일은 보관 기간 정리 작업이 맡습니다 |
| 이름, 이메일, 본문을 폼에서 뺌 | 사용자가 보낸 원문은 관리자가 고치지 않습니다 |

| 선택 | 이유 |
|---|---|
| 이메일로 답변을 보내지 않음 | 메일 발송 기능이 아직 없습니다. 답변은 이 화면에만 기록됩니다 (나중에 필요해지면 `after_model_change` 후크에서 보낼 수 있습니다) |

## 6. 시험 (`tests/test_admin_inquiry.py`, 12개)

| 시험 | 결과 |
|---|---|
| 새로 만들기와 삭제가 모두 막혀 있음 | 통과입니다 |
| 목록 열 5개, 번호 내림차순, 이메일과 제목 검색 | 통과입니다 |
| 수정 폼이 답변과 상태 두 칸뿐 | 통과입니다 |
| 새 답변: `answered_at`이 지금 UTC(시간대 없음), 상태 `RECEIVED` → `ANSWERED` | 통과입니다 |
| 상태가 열거형이 아니라 글자로 와도 같음 | 통과입니다 |
| 답변을 고침: 시각은 새로 정하되 `CLOSED`는 유지 | 통과입니다 |
| 같은 답변(앞뒤 공백만 다름 포함): 시각을 건드리지 않음 | 통과입니다 |
| 답변을 비움(빈 글자, 공백, `None`): `answered_at`이 비고 상태는 그대로 | 통과입니다 |
| 상태만 바꿈: 다른 값은 그대로 | 통과입니다 |

12개 모두 통과했습니다. `ruff`도 통과했습니다.

# Backend(FastAPI/Python) 91 — `UserAdmin` (회원 보기 화면 설정)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/admin.py` |
| 작업 | P1-21 공지 API와 관리자 화면: `/admin`에서 회원을 보고 정지, 관리자 지정을 하는 화면의 설정 (PRD 01 FR-115) |
| 작성 | Claude (AI), 2026-10-13, 사용자 검토 |
| 이전 · 다음 | 90 `InquiryAdmin` · 92 `setup_admin()` (다음) |

## 1. 하는 일

회원 목록을 보고, 필요하면 **역할(`role`)과 상태(`status`)만** 바꾸는 읽기 위주의 화면입니다.

| 영역 | 설정 |
|---|---|
| 만들기, 삭제 | **둘 다 없음.** 가입은 join 화면에서, 탈퇴는 회원 본인이 합니다 |
| 목록 열 | 번호, 이메일, 닉네임, 역할, 상태, 마지막 로그인, 가입일. 최근 가입이 위 |
| 검색 | 이메일, 닉네임 |
| `password_hash` | **목록, 상세, 폼, CSV 내보내기 어디에도 나오지 않음** |
| 수정 폼 | 역할과 상태 두 칸만 (정지 처리, 관리자 지정용) |
| 막는 것 1 | 상태를 **탈퇴(`DELETED`)로 바꾸기** |
| 막는 것 2 | 로그인한 관리자가 **자기 자신**을 관리자에서 내리거나 정지하기 |

두 가지를 막을 때는 저장이 되지 않고, 수정 화면에 영어 오류 문구가 나옵니다.

| 막은 경우 | 화면에 나오는 문구 |
|---|---|
| `DELETED`로 바꾸기 | "Use the member's own withdrawal instead of setting DELETED here." |
| 자기 자신을 내리거나 정지 | "You cannot demote or suspend your own account." |

## 2. 언제 쓰나

| 호출하는 곳 | 화면 |
|---|---|
| `setup_admin()`으로 등록된 뒤 (92번) `/admin/user/…` | 회원 목록, 상세, 역할과 상태 수정 |

## 3. 코드

**import** (이번에 추가한 것)

```
from app.models.user import UserRole, UserStatus   # UserStatus 추가
```

**설정과 함수**

```
def enum_text(value) -> str:
    return str(getattr(value, "value", value))


class UserAdmin(ModelView, model=User):
    can_create = False
    can_delete = False
    column_list = [User.user_id, User.email, User.nickname, User.role,
                   User.status, User.last_login_at, User.created_at]
    column_default_sort = [("user_id", True)]
    column_searchable_list = [User.email, User.nickname]
    column_details_exclude_list = [User.password_hash]
    column_export_exclude_list = [User.password_hash]
    form_columns = [User.role, User.status]

    async def on_model_change(self, data, model, is_created, request):
        status = enum_text(data.get("status", model.status))
        role = enum_text(data.get("role", model.role))
        if status == UserStatus.DELETED and model.status != UserStatus.DELETED:
            raise ValueError("Use the member's own withdrawal instead of setting DELETED here.")
        if model.user_id is not None and model.user_id == request.session.get("admin_user_id"):
            if role != UserRole.ADMIN or status != UserStatus.ACTIVE:
                raise ValueError("You cannot demote or suspend your own account.")
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

### 문법 풀이

| 식 | 뜻 |
|---|---|
| `column_list` | 목록에 보일 열입니다. 여기에 없는 `password_hash`는 목록에 나오지 않습니다 |
| `column_details_exclude_list` | 상세 화면에서 뺄 열입니다 (목록과 달리 상세는 기본이 모든 열이라 따로 빼야 합니다) |
| `column_export_exclude_list` | CSV 내보내기에서 뺄 열입니다. 이것도 기본이 모든 열이라 따로 빼야 합니다 |
| `form_columns` | 폼에 넣을 칸입니다. `password_hash`와 다른 칸은 수정할 수 없습니다 |
| `enum_text(value)` | 열거형(`UserRole.ADMIN`)이든 글자(`"ADMIN"`)든 같은 글자로 읽습니다. `getattr(값, "value", 값)`은 `value` 속성이 있으면 그것을, 없으면 값 자체를 씁니다 |
| `data.get("status", model.status)` | 폼에 상태가 있으면 그 값, 없으면 지금 저장된 값입니다 |
| `raise ValueError(문구)` | 저장을 중단합니다. SQLAdmin이 오류를 잡아 수정 화면 위에 문구를 보여 줍니다 |
| `request.session.get("admin_user_id")` | 86번 로그인이 저장한 지금 로그인한 관리자의 번호입니다 |

## 4. 흐름

회원의 역할이나 상태를 저장할 때:

1. 폼의 값(없으면 지금 값)을 글자로 읽습니다.
2. 상태를 `DELETED`로 바꾸려 하면(원래 `DELETED`가 아니었다면) 막습니다.

3. 지금 수정하는 회원이 로그인한 관리자 본인이고 결과가 "ADMIN이 아님" 또는 "ACTIVE가 아님"이면 막습니다.
4. 통과하면 SQLAdmin이 저장합니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| `password_hash`를 **네 곳 모두**에서 뺌 | 목록은 `column_list`로 이미 안 나오지만, 상세와 CSV 내보내기는 기본이 모든 열입니다. 하나라도 빠뜨리면 비밀번호 해시가 화면이나 파일로 나갑니다. 시험에서 네 곳 모두 확인합니다 |
| 폼을 역할과 상태 두 칸으로 | 닉네임, 이메일, 저장 용량을 관리자가 바꾸는 일은 없고, 바꿔야 하면 DB에서 직접 합니다 |
| `DELETED`로 바꾸기를 막음 | 탈퇴는 이메일 해제(3일 재가입 제한)와 보관 정리와 얽혀 있어서(P1-09), 상태 값만 바꾸면 그 절차가 빠집니다 |
| 이미 `DELETED`인 회원을 그대로 저장하는 것은 막지 않음 | 상태를 바꾸려는 것이 아니기 때문입니다 |
| 자기 자신을 내리거나 정지하는 것을 막음 | 마지막 관리자가 실수로 자기를 내리면 아무도 `/admin`에 들어올 수 없게 되고, DB에서 직접 고쳐야 합니다 |
| 다른 관리자를 내리는 것은 허용 | 관리자끼리 권한을 조정할 수 있어야 합니다 |
| 목록에서 삭제 버튼을 없앰 | 회원 삭제는 탈퇴 절차(소프트 삭제와 정리 작업)로만 합니다 |

## 6. 시험 (`tests/test_admin_user.py`, 14개)

| 시험 | 결과 |
|---|---|
| 만들기와 삭제가 막혀 있음 | 통과입니다 |
| `password_hash`가 목록, 상세, 폼, 내보내기 네 곳 어디에도 없음 | 통과입니다 |
| 상세에는 이메일, 닉네임, 역할, 상태, 마지막 로그인이 있음 | 통과입니다 |
| 목록 열 7개, 번호 내림차순, 이메일과 닉네임 검색 | 통과입니다 |
| 수정 폼이 역할과 상태 두 칸뿐 | 통과입니다 |
| 다른 회원을 정지, 다른 회원을 관리자로 지정 | 허용됨 (값이 글자로 와도 같음) |
| `DELETED`로 바꾸기 (열거형과 글자 두 경우) | `ValueError`, 문구에 "withdrawal" |
| 이미 `DELETED`인 회원을 그대로 저장 | 막지 않음 |
| 자기 자신을 `USER`로 내리거나 `SUSPENDED`로 정지 | `ValueError`, 문구에 "your own account" |
| 자기 자신을 그대로 저장, 폼에 값이 없을 때 지금 값 사용 | 통과입니다 |

14개 모두 통과했습니다. `ruff`도 통과했습니다.

# Backend(FastAPI/Python) 92 — `setup_admin()` (관리자 화면 붙이기)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/admin.py`, `app/core/config.py`, `studionx/.env.example` |
| 작업 | P1-21 공지 API와 관리자 화면: 만든 로그인과 세 화면을 `/admin`에 실제로 연결 (PRD 01 FR-115) |
| 작성 | Claude (AI), 2026-10-13, 사용자 검토 |
| 이전 · 다음 | 91 `UserAdmin` · (P1-21 끝, 실제 화면 시험은 개요 문서에서) |

## 1. 하는 일

86~91번에서 만든 부품을 한 곳에 모아 **SQLAdmin 관리자 화면**을 서버에 붙입니다. `main.py`가 이미 이 함수를 부르고 있어서(정적 파일 마운트보다 앞), 이 함수를 채우면 서버가 켜질 때 `/admin`이 생깁니다.

| 항목 | 내용 |
|---|---|
| 주소 | 설정 `admin_path` (기본 `/admin`) |
| 로그인 | `AdminAuth` (ADMIN 계정만, 세션 쿠키 서명은 `jwt_secret`) |
| 화면 | 공지(`DevNoteAdmin`), 문의(`InquiryAdmin`), 회원(`UserAdmin`) |
| 제목 | "Studio Nx Admin" |

**주소를 설정으로 뺐습니다 (사용자 결정, 2026-10-13).** 개발 중에는 `/admin` 그대로 두고, 배포할 때 `.env`의 `ADMIN_PATH`를 추측하기 어려운 값으로 바꿉니다.

```
# .env (배포 때)
ADMIN_PATH=/ops-7f3a9c1e
```

| `ADMIN_PATH` 값 | 결과 |
|---|---|
| `/admin` (기본) | `/admin/login` |
| `/ops-7f3a9c1e` 또는 끝에 `/`가 있는 `/ops-7f3a9c1e/` | `/ops-7f3a9c1e/login` (`/admin`은 열리지 않음) |
| `admin`(슬래시 없음), `/`, 빈 값, `/api/v1…` | 서버가 켜질 때 `ValueError` |

## 2. 언제 쓰나

| 호출하는 곳 | 시점 |
|---|---|
| `main.py`의 `setup_admin(app)` | 서버가 켜질 때 한 번 |

## 3. 코드

**import** (이번에 추가한 것)

```
from sqladmin import Admin, ModelView
from app.core.config import get_settings
from app.db.session import SessionLocal, engine
```

**설정** (`app/core/config.py`)

```
admin_path: str = "/admin"
```

**함수**

```
def setup_admin(app: FastAPI) -> Admin:
    settings = get_settings()
    path = settings.admin_path
    if (
        not path.startswith("/")
        or len(path) < 2
        or path.rstrip("/").startswith(settings.api_prefix)
    ):
        raise ValueError(f"ADMIN_PATH is not usable: {path!r}")
    admin = Admin(
        app,
        engine,
        base_url=path.rstrip("/"),
        authentication_backend=AdminAuth(secret_key=settings.jwt_secret),
        title="Studio Nx Admin",
    )
    admin.add_view(DevNoteAdmin)
    admin.add_view(InquiryAdmin)
    admin.add_view(UserAdmin)
    return admin
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

## 문법 풀이

| 식 | 뜻 |
|---|---|
| `Admin(app, engine, ...)` | `app`에 SQLAdmin을 붙입니다. `engine`은 DB 연결이며, 이 프로젝트는 동기 엔진이라 `session_maker`도 동기 방식입니다 |
| `base_url=path.rstrip("/")` | 관리자 화면의 주소입니다. 끝의 `/`는 지웁니다 |
| `authentication_backend=AdminAuth(...)` | 로그인 방식입니다. 이것이 있으면 모든 `/admin/…` 요청에서 `authenticate`(87번)가 먼저 불립니다 |
| `secret_key=settings.jwt_secret` | 세션 쿠키 서명용 비밀값입니다. `.env`의 `JWT_SECRET`을 함께 씁니다 |
| `admin.add_view(클래스)` | 화면 하나를 등록합니다. 주소는 `/admin/dev-note/list`처럼 모델 이름으로 정해집니다 |
| `path.rstrip("/").startswith(settings.api_prefix)` | 관리자 주소가 API 주소 아래에 들어가는 실수를 막습니다 |
| `-> Admin` | 만든 `Admin` 객체를 돌려줍니다. 시험이 등록된 화면을 확인하는 데 씁니다 |

## 4. 흐름

1. 설정에서 관리자 주소를 읽고, 쓸 수 없는 값이면 서버를 켜지 않습니다.
2. `Admin`을 만들어 `app`에 `/admin`(또는 설정한 주소)으로 붙입니다.
3. 화면 셋을 등록합니다.
4. 이후 `/admin/…` 요청은 모두 `AdminAuth.authenticate`를 거치고, 통과하지 못하면 로그인 화면으로 갑니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 주소를 설정값 `ADMIN_PATH`로 | 배포할 때 코드 수정 없이 `.env`만 바꿔서 주소를 숨길 수 있습니다 |
| 잘못된 주소는 서버를 켤 때 오류 | 배포 설정 실수를 운영 중에 알게 되는 것보다 시작할 때 바로 알아야 합니다 |
| IP 제한은 두지 않음 | 서버(앱) 안에서 IP를 믿고 거르는 것은 프록시 설정에 달려 있어서 위험하고, 앞단(Nginx 등)에서 거는 것이 맞습니다. 배포 준비 때 정합니다 |
| `jwt_secret`을 세션 서명에 같이 씀 | 비밀값을 하나만 관리합니다. 서명용이라 값이 달라도 API 토큰과 쿠키가 서로 통하지 않습니다 |
| `Admin`을 돌려줌 | 시험이 화면 등록과 인증 방식을 확인할 수 있습니다 |
| 앞으로 `JWT_SECRET`이 `change-me`인 채 운영하면 안 됨 | 세션 쿠키 서명이 추측되기 때문입니다. `.env.example`에 이미 안내가 있고, 배포 준비 때 다시 확인합니다 |

## 6. 시험 (`tests/test_setup_admin.py`, 11개)

| 시험 | 결과 |
|---|---|
| 작은 앱에 붙이면 화면 3개(`dev-note`, `inquiry`, `user`)와 `AdminAuth`가 등록되고 `/admin`이 생김 | 통과입니다 |
| `ADMIN_PATH=/ops-7f3a9c1e/`이면 `/ops-7f3a9c1e/login`은 열리고 `/admin/login`은 404 | 통과입니다 |
| `admin`, `/`, 빈 값, `/api/v1`, `/api/v1/admin` | 모두 `ValueError` |
| 실제 앱: 로그인하지 않고 `/admin/`, 공지, 문의, 회원 목록 | 모두 로그인 화면으로 이동 |
| 실제 앱: `/admin/login` | 200, "Studio Nx Admin" |
| 실제 앱: 없는 계정으로 로그인 | 400, 로그인 쿠키 없음 |
| 실제 앱: API, 홈 화면, 없는 페이지는 그대로 | 통과입니다 |

11개 모두 통과했습니다. 백엔드 전체 pytest와 `ruff`도 통과했습니다. 로그인한 뒤의 화면 흐름은 서버를 띄워서 확인합니다 (개요 문서의 시험).